

Containers - Docker

Technologies de conteneurisation et exploitation



Sommaire

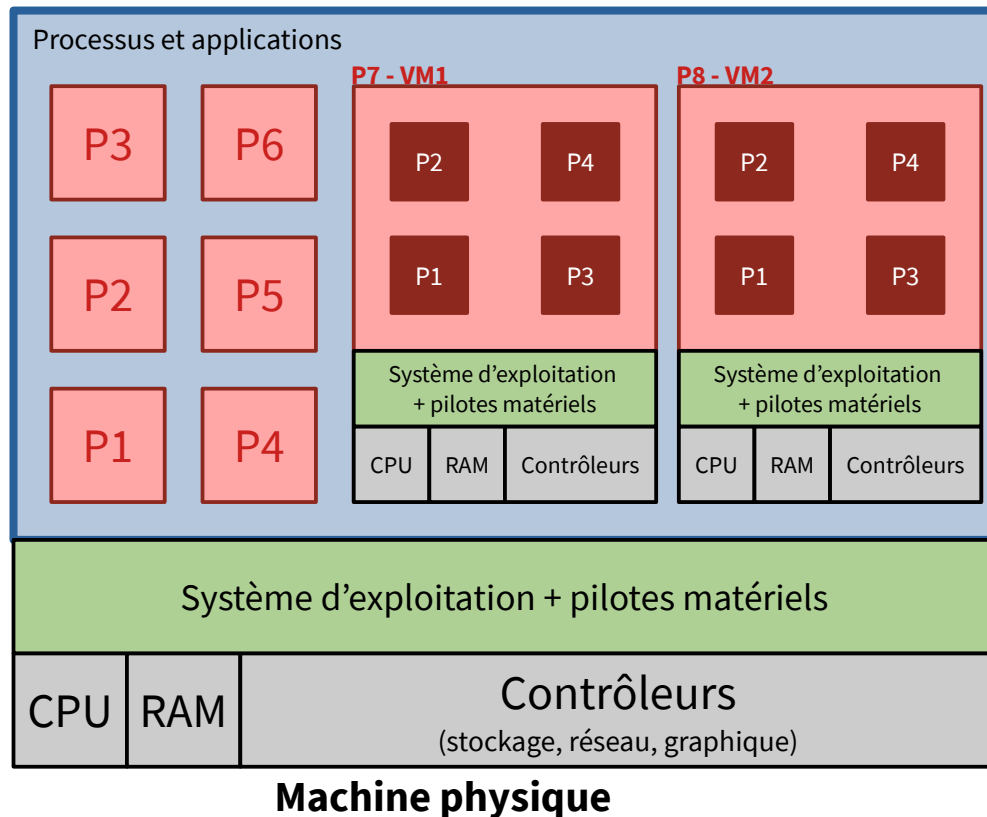
- Principes de la conteneurisation
- Notions sur les containers LXC
- Containers Docker
- Généralités sur les orchestrateurs de containers

Principes – Rappels sur la virtualisation

Containers - Virtualisation

Virtualisation par hyperviseur

- Un processus de la machine physique émule :
 - CPU (émulateur)
 - RAM
 - Contrôleurs (stockage, réseau, graphique...)
- Un système d'exploitation complet (noyau, pilotes, environnement...) est installé dans le processus.
- Les applications natives du système invité fonctionnent dans la machine virtuelle.



Containers - Virtualisation

Avantages

- Possibilité de virtualiser n'importe quel système d'exploitation.
- Machine virtuelle entièrement indépendante du matériel physique.
- Technologies avancées de gestion des machines virtuelles (migration à chaud, infrastructures de virtualisation)
- Isolation parfaite entre l'hôte et l'invité

Inconvénients

- Ressources CPU « gaspillées » par l'émulation.
- Ressources mémoires à utiliser importantes
- Espace disque à utiliser important

Containers - Virtualisation

Technologies

- Hyperviseurs de type 1 (OS spécialisé) : VMWare ESXi, Microsoft Hyper-V, Linux KVM, Linux Xen...
- Hyperviseurs de type 2 (OS générique) : Oracle Virtualbox, VMWare Workstation, Parallels Desktop...

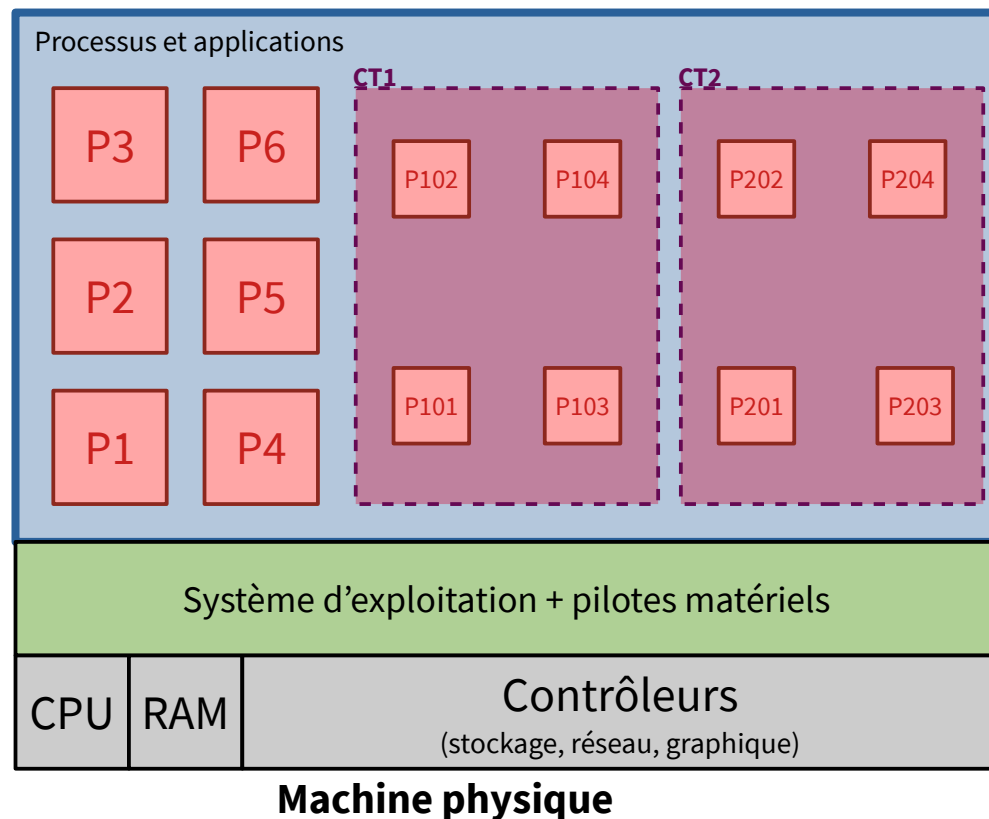


Principes de l'isolation de processus

Containers – Isolation de processus

Isolation de processus

- Les processus d'un container peuvent communiquer entre eux mais pas avec ceux d'un autre container ou du système hôte.
- Le noyau du système hôte est utilisé par les processus des containers.
- Aucun matériel émulé n'est utilisé.
- Sur Linux : utilisation des « namespaces ».



Containers – Isolation de processus

Avantages

- Pas d'émulation donc pas de « gaspillage » CPU.
- Très faible utilisation mémoire : seuls les processus nécessaires au fonctionnement du container sont chargés en mémoire.
- Très bonnes performances.

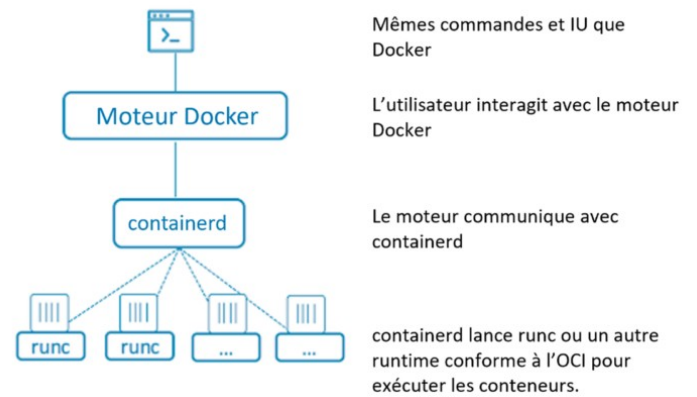
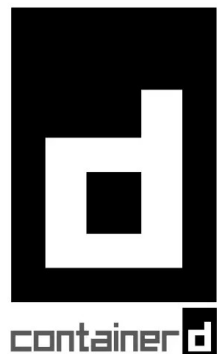
Inconvénients

- Partage du noyau hôte donc le système invité doit faire tourner le même système que sur l'hôte.
- Certaines fonctionnalités proches du noyau difficilement utilisables.
- Isolation entre le système hôte et invité parfois imparfaite.

Containers – Isolation de processus

Un système de conteneurisation est composé de :

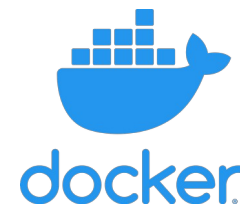
- **Runtime de container** : composant logiciel permettant de gérer le fonctionnement d'un container (démarrage, arrêt, stockage...) → containerd, cri-o
- **Moteur de container** : composant logiciel permettant d'interagir avec un moteur de container → docker, podman



Containers – Isolation de processus

Technologies

- **Containers applicatifs (intègre uniquement un service)**
 - Windows Containers
 - Docker
 - Podman
- **Containers systèmes (intègrent un gestionnaire de démarrage et de service)**
 - FreeBSD Jails
 - Oracle Solaris Containers
 - Linux OpenVz (obsolète)
 - Linux LXC



podman



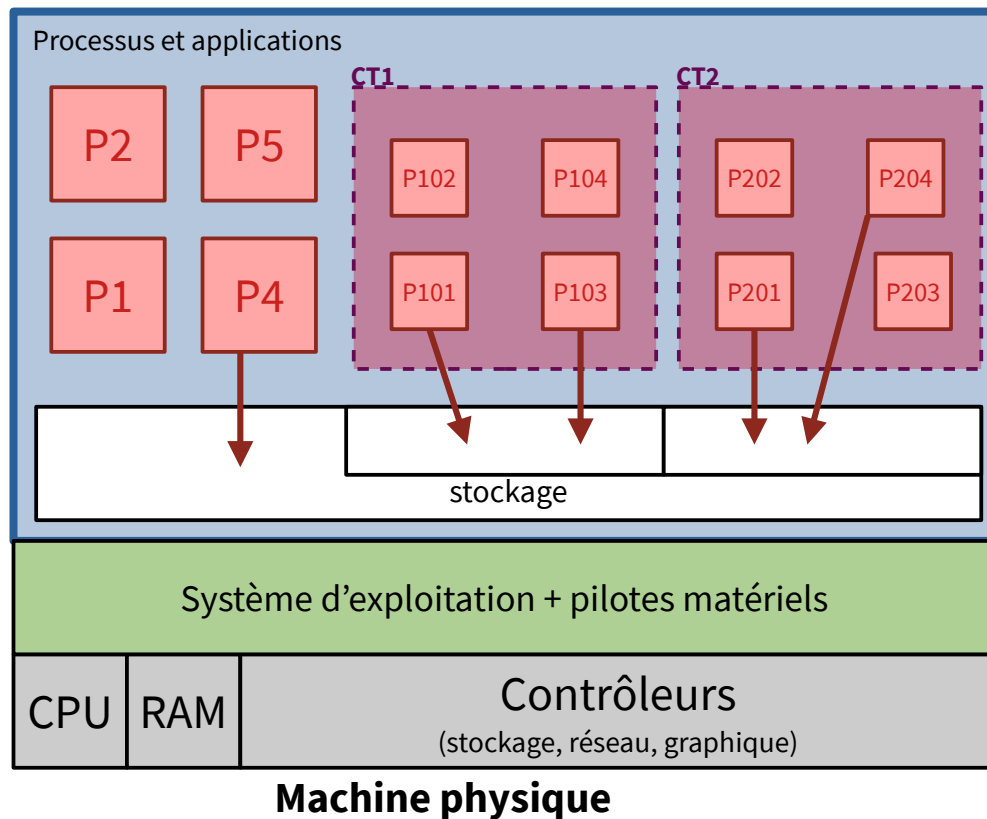
Gestion du stockage des containers

Containers - Stockage

L'isolation de processus participe à la création de containers en s'assurant que les processus d'un container ne puissent pas communiquer avec les processus d'un autre container

Le stockage d'un container doit aussi être isolé des autres containers.

Il faut pouvoir limiter l'espace disque utilisé.



Containers - Stockage

Techniques d'isolation du stockage

- **chroot** : modification de la racine système (/) pour un ensemble de processus (pas de limitation de l'espace disque)
- **Point de montage bind** : monter un dossier dans un autre dossier du système (pas de limitation de l'espace disque)
- **Image disque** : fichier image monté dans l'arborescence du container (issue de la virtualisation)
- **LVM** : utilisation d'un volume logique comme espace de stockage
- **ZFS** : utilisation d'un système de fichier ZFS (quota pour limiter l'espace disque)
- **BTRFS** : utilisation d'un sous-volume BTRFS (quota pour limiter l'espace disque)

Ces techniques sont plus ou moins bien intégrées dans les gestionnaires de containers.

Gestion du réseau sur les containers

Containers - Réseau

Chaque container doit disposer de sa propre interface réseau.

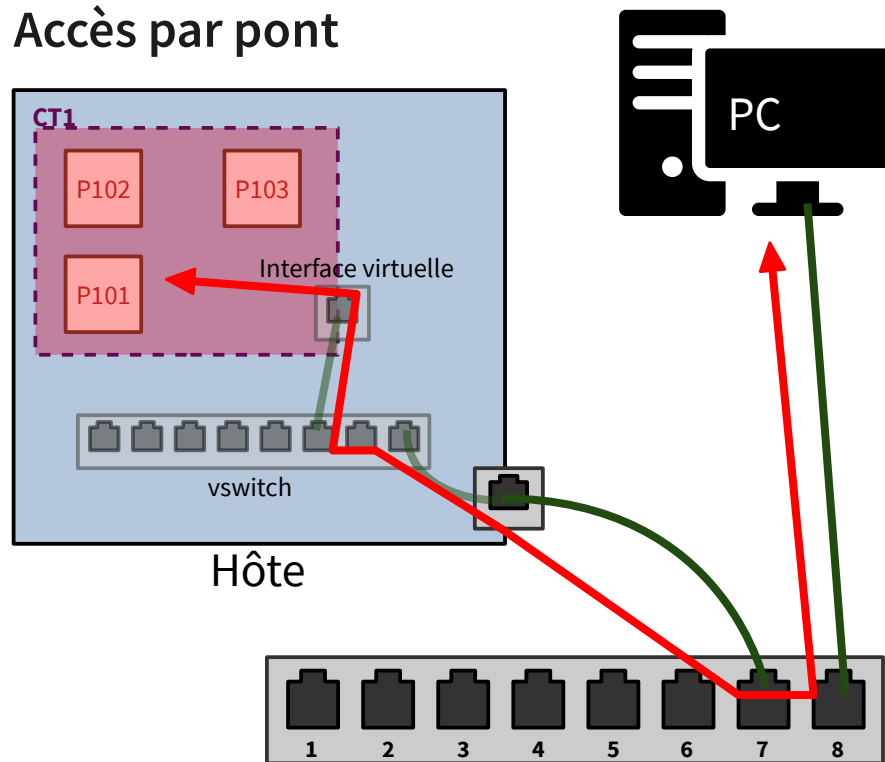
- La communication avec les hôtes, les machines physiques et les autres containers passe par le réseau
- Utilisation d'interfaces réseaux virtuelles attachées aux containers
- Utilisation de switchs virtuels (brctl, openvswitch) sur lesquelles sont connectées les interfaces virtuelles
- Utilisation de Netfilter / iptables pour filtrer le trafic réseau et mettre en place des accès NAT.

Type d'accès

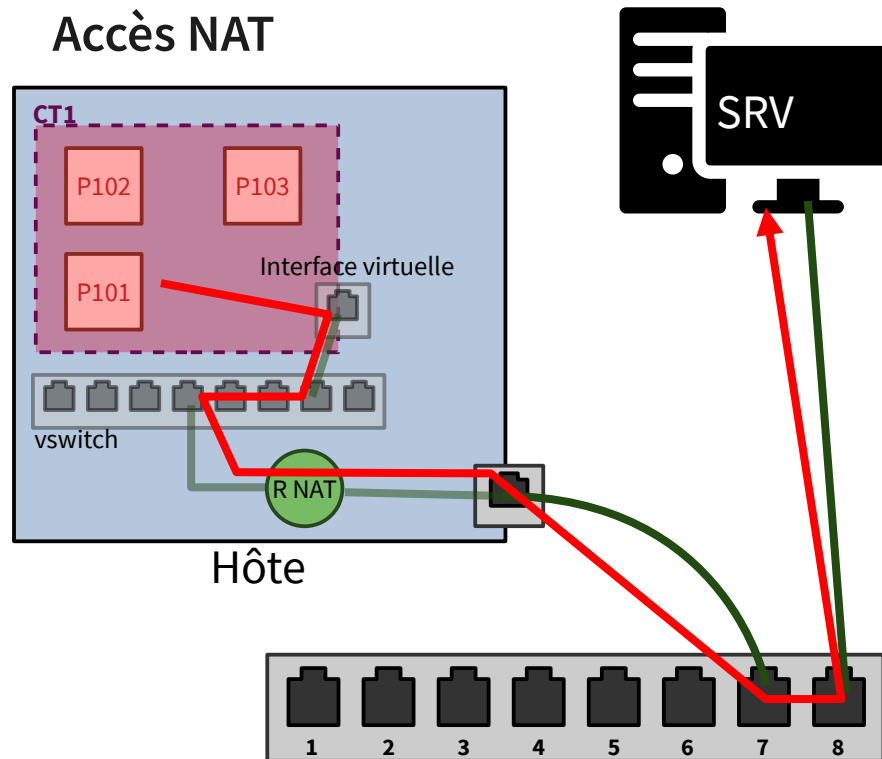
- Accès par pont : accès aux containers depuis le réseau physique comme une machine du réseau
- Accès NAT : accès aux containers par redirection de ports
- Accès hôte : le processus du container est lié à des ports de l'interface de l'hôte

Containers - Réseau

Accès par pont



Accès NAT

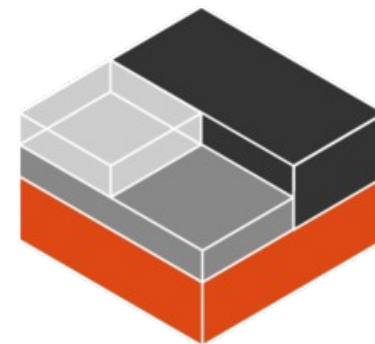


LXC Linux Containers

Containers - LXC

Création et gestion de containers

- Intégré au noyau Linux depuis la version 2.6.24
- Utilisable sur toute distribution GNU/Linux récente
- Permet de créer des containers de **systèmes** GNU/Linux (Ubuntu, Debian, CentOS...)
- Destiné à concurrencer les machines virtuelles par hypervision
- Utilise les **namespaces** et les **cgroups** pour l'isolation et la gestion des processus isolés
- Utilise des « **templates** » pour la création des containers
- Utilisé par **Proxmox VE** en parallèle de KVM



Containers - LXC

<code>lxc-create</code>	Création d'un container à partir d'un template <code>lxc-create -t download -n vm1</code> <code>lxc-create -t download -n vm2 -- -d ubuntu -r groovy -a amd64</code>
<code>lxc-ls</code>	Liste les containers présents sur le système <code>lxc-ls -f</code>
<code>lxc-start</code>	Démarre un container <code>lxc-start vm1</code>
<code>lxc-attach</code>	Connexion à la console d'un container <code>lxc-attach vm1</code>
<code>lxc-stop</code>	Arrête un container <code>lxc-stop vm1</code>
<code>lxc-destroy</code>	Supprime un container <code>lxc-destroy vm1</code>

Containers - LXC

Fichiers et dossiers de configuration

<code>/etc/lxc/default.conf</code>	Paramètres par défaut à la création d'un container
<code>/etc/lxc/lxc-usernet</code>	Configuration réseau des containers
<code>/usr/share/lxc/templates</code>	Dossier des templates LXC utilisé à la création
<code>/var/lib/lxc/</code>	Dossier racine des containers
<code>/var/lib/lxc/vm1/</code>	Dossier du container <code>vm1</code>
<code>/var/lib/lxc/vm1/config</code>	Fichier de configuration du container <code>vm1</code>
<code>/var/lib/lxc/vm1/rootfs/</code>	Dossier contenant la racine du container <code>vm1</code>

Demo time...



Conteneurisation d'applications

Containers - Docker

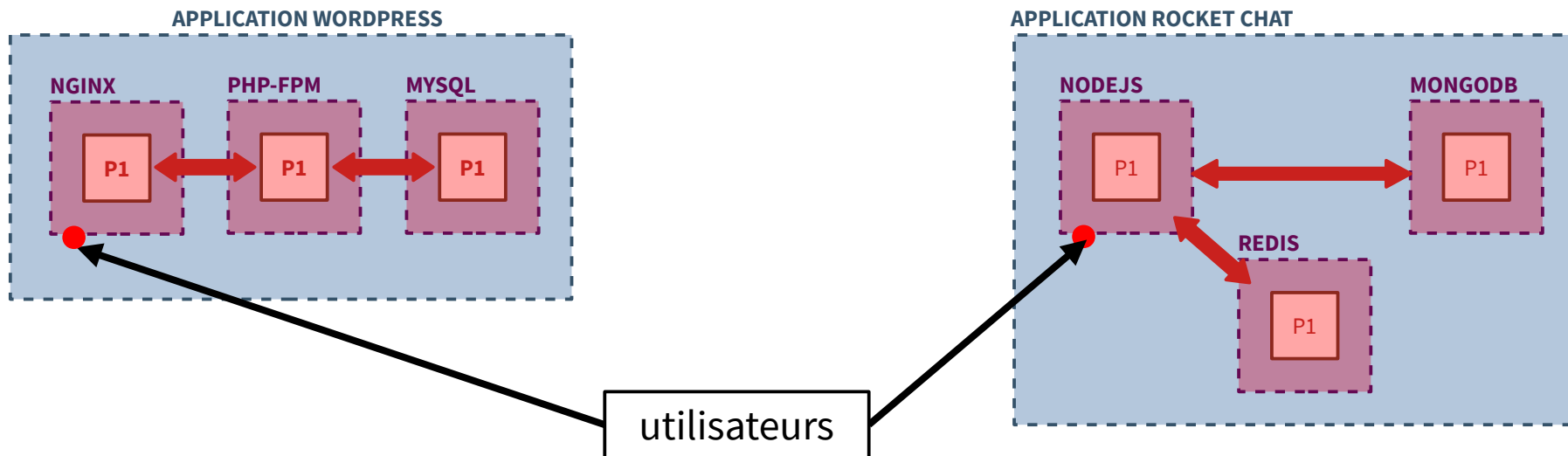
Docker – Conteneurisation d'applications : un nouveau paradigme

- Permet une utilisation simple et efficace des containers pour exécuter des applications dans des environnements conteneurisés.
- Destiné à être utilisé :
 - Par les **développeurs** afin de disposer facilement d'un environnement conforme à celui mis en production
 - Par les **administrateurs systèmes** afin de déployer des applicatifs efficacement
 - Par les **devops** afin de permettre une intégration et un déploiement continus d'applications

Principe initial

Exécuter un service par container

Containers - Docker

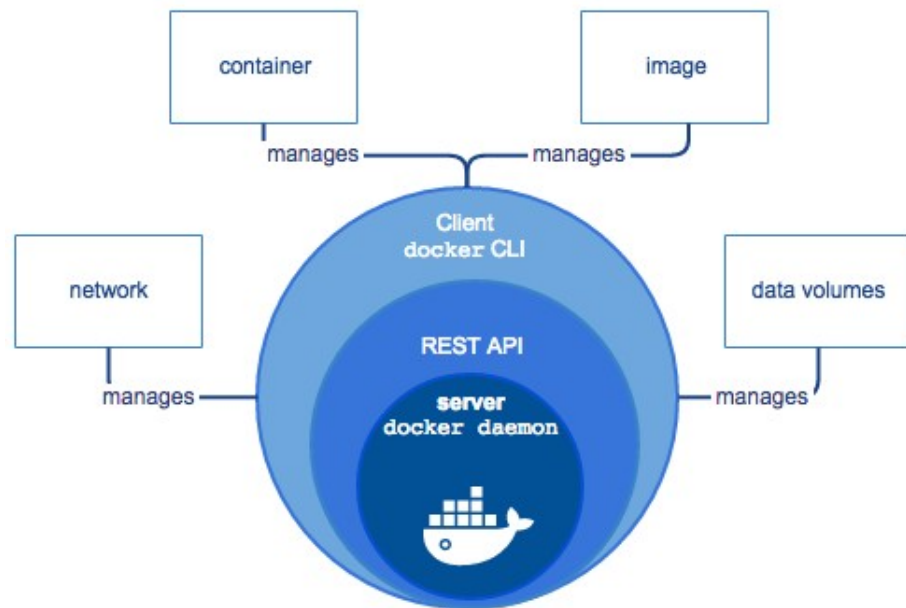


- Une application est composée de plusieurs containers faisant tourner un service chacun.
- Les containers d'une même application peuvent communiquer par un réseau virtuel
- L'application est accessible depuis le réseau physique par certains ports publiés

Containers - Docker

Docker – Écosystème

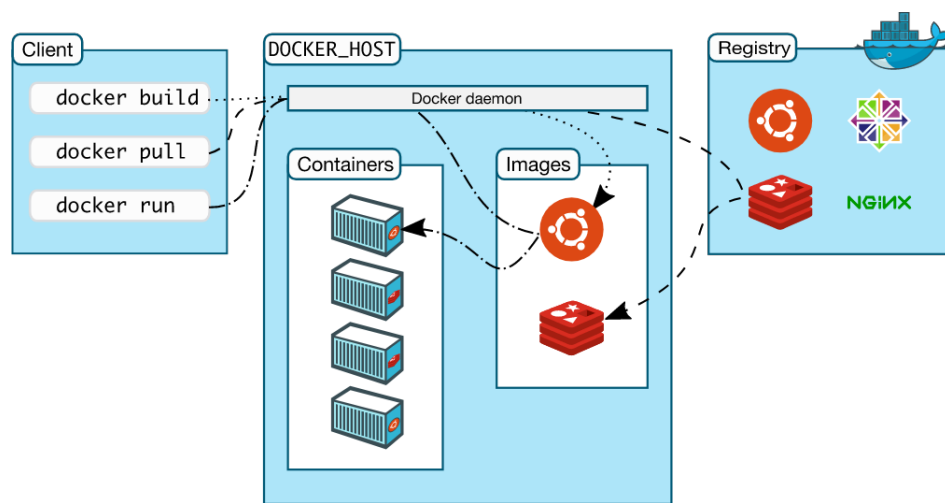
- Projet open source depuis 2013
- Version entreprise payante : cédée à Mirantis en 2019. Remplacée par des offres **Pro / Team / Business**
- 1 service, 1 API, 1 client CLI pour gérer les containers Docker
- Écosystème logiciel important
- Docker Desktop disponible sur Linux, Windows, MacOS X
- Certains concepts inspirés de git / github (pull, push, commit, dépôts...)



Containers - Docker

Docker – Une architecture originale

- Le client interagit avec le **docker daemon** à travers la commande **docker** ou des applications tierces à travers l'API.
- Le **docker daemon** gère l'instanciation des containers (isolation, ressources, stockage, réseau) en relation avec un **container runtime** (containerd).
- Le container est créé à partir d'une **image**.
- Les images peuvent être récupérées et stockées sur des **registry** (Docker Hub, privé...)



Docker – Création de containers

Containers - Docker

`docker run` permet de créer un container à partir d'une image et de l'exécuter.

Ce container exécute un processus défini à la création de l'image ou précisé à l'exécution de la commande `docker run`.

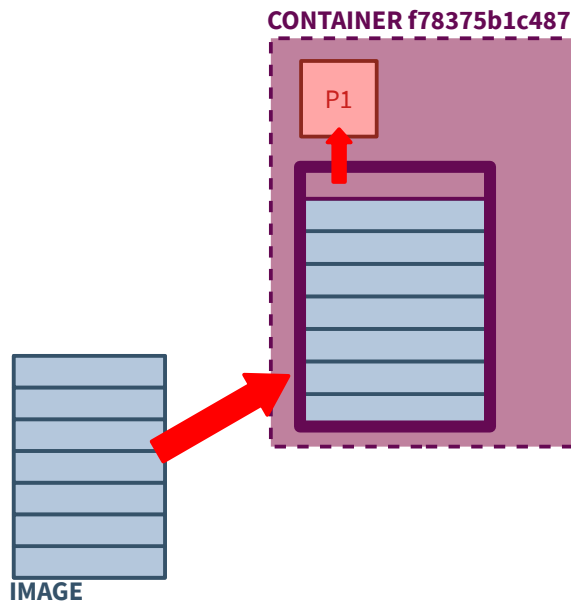
```
docker run hello-world  
           image
```

```
docker run ubuntu cat /etc/issue  
           image    commande
```

Quand plus aucun processus ne tourne dans le container, celui-ci s'arrête.

`docker ps` et `docker ps -a` permettent de lister les containers.

Les containers sont identifiés par un `UUID` et un nom.



Containers - Docker

Arguments de `docker run`

- `-i` : connecte `stdin` à la console, permet l'interaction avec le container par le clavier.
- `-t` : alloue un pseudo-terminal pour utiliser les fonctionnalités du terminal de l'hôte.
- `--name` : précise un nom unique pour le container. Sans cet argument, un nom est choisi automatiquement.
- `-d` : détache le container du terminal afin qu'il tourne en tâche de fond.
- `--rm` : supprime le container dès qu'il s'arrête.
- `--restart` : mode de redémarrage (`no`, `always`, `unless-stopped`, `on-failure`)
- `-e` : permet de spécifier une variable d'environnement

```
docker run -it -name web --rm httpd
```

```
docker run -d --name web --restart unless-stopped httpd
```

Containers - Docker

Commandes docker en relation avec l'exécution des containers

- `docker stop` : arrête un container en cours d'exécution (ne le supprime pas).
- `docker start` : démarre un container arrêté.
- `docker rm` : supprime un container arrêté (ou démarré avec `-f`)
- `docker exec` : exécute une commande dans un container en cours d'exécution
- `docker logs` : affiche les informations retournées sur `stdout` et `stderr` du processus exécuté dans le container.
- `docker stats` : affiche des infos sur les ressources consommées par les containers
- `docker top` : affiche les processus s'exécutant dans un container
- `docker inspect` : affiche des informations détaillées sur la composition d'un container

```
docker exec -it -name web bash
docker logs web
docker rm -f web
```

Docker – Gestion du stockage

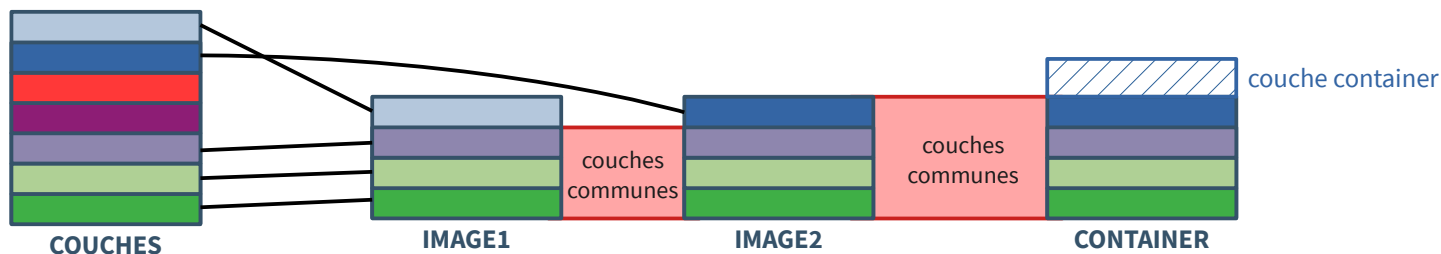
Containers - Docker

Le stockage des containers s'appuie sur des images et un système de couches.

Le système de couche est géré par le système hôte : **aufs**, **btrfs**, **zfs**...

Le système de couche permet :

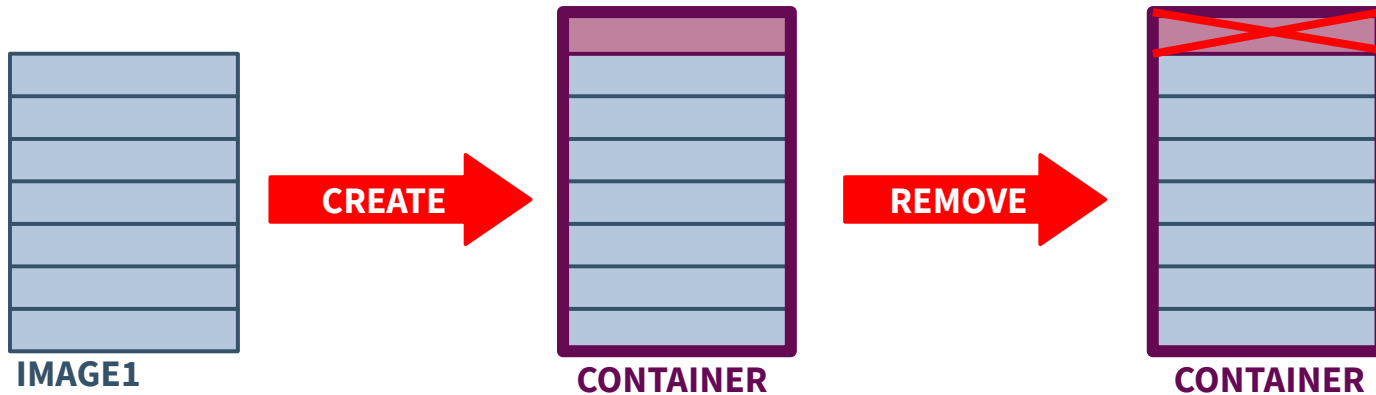
- Dé-dupliquer les couches identiques de plusieurs images.
- Ne transférer que les couches inexistantes vers une destination (stockage local, dépôt...).
- Différencier le stockage de l'image du stockage propre au container.



Containers - Docker

Le stockage propre à un container n'est pas persistant.

A la suppression du container, ce stockage est perdu.

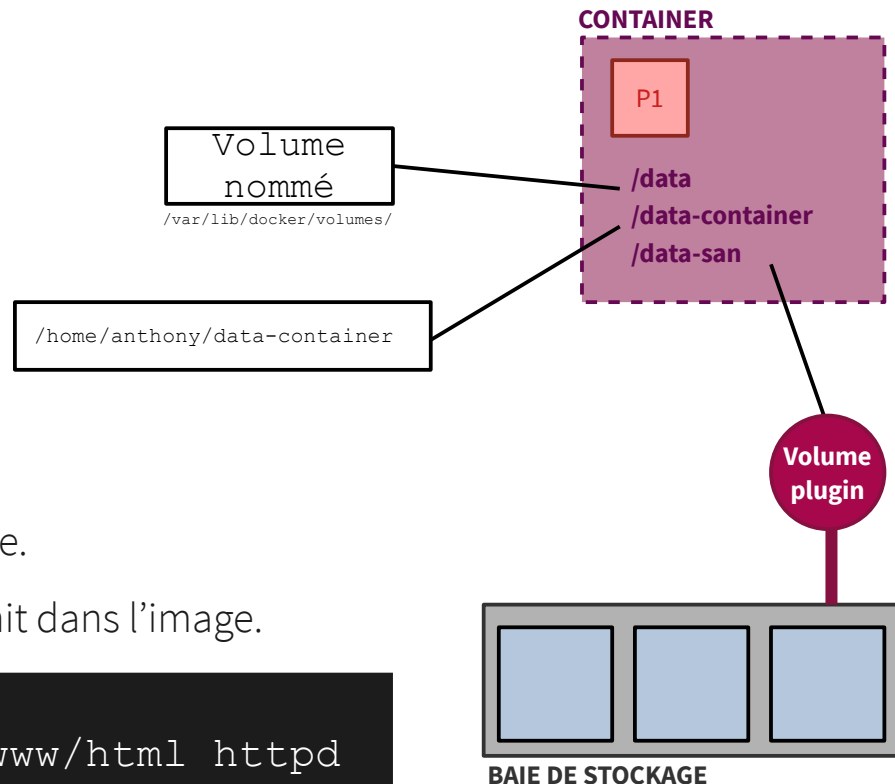


Containers - Docker

Volumes : données persistantes

- Données conservées après la suppression du container.
- Peut être utilisé dans différents containers.
- Utilisation de montages **bind**
 - Dossier local
 - Volume nommé
 - Volume plugin pour une technologie de stockage
- Il est possible de monter autant de volumes que nécessaire.
- Un volume peut être créé automatiquement s'il a été défini dans l'image.

```
docker run -v data:/var/www/html httpd
docker run -v /home/anthony/project:/var/www/html httpd
```



Docker – Gestion du réseau

Containers - Docker

Docker utilise des briques systèmes classiques pour la gestion du réseau mais propose une interface simple pour sa gestion.

- `docker network` permet de créer, supprimer, configurer les réseaux docker.
- Il est possible de choisir un réseau docker pour un container avec le paramètre `--network`.
- Un réseau docker intègre un serveur DHCP délivrant des adresses pour un réseau IP privé choisit manuellement ou automatiquement.
- Les containers attachés au même réseau docker peuvent communiquer directement.
- Les containers attachés à des réseaux docker différents ne peuvent pas communiquer directement.
- `docker inspect` permet d'afficher les paramètres réseaux d'un container en cours d'exécution.

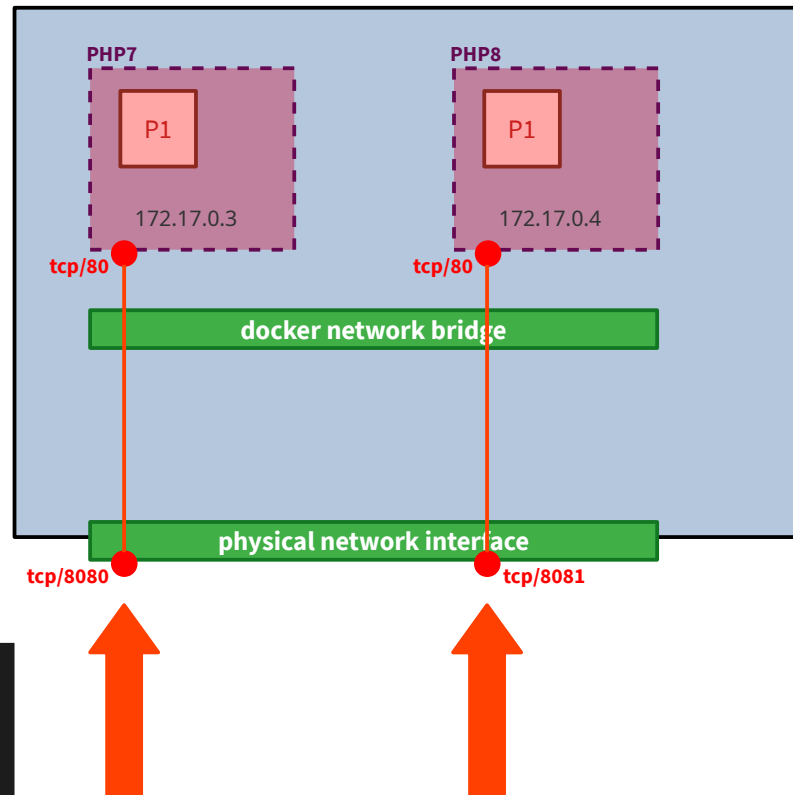
```
docker network create --driver bridge --subnet 172.20.0.0/16 lan1
docker run -d --network lan1 httpd
docker run -d --network lan1 mysql
```

Containers - Docker

Réseaux **bridge** (vswitch + NAT)

- Le plus utilisé avec Docker.
- Permet d'accéder aux container par des redirections de port.
- Permet de faire tourner autant de containers que nécessaire écoutant sur un même port (IP propre à chaque container).
- Trafic routé entre le container et le réseau physique.
- -p permet de créer une redirection de port
- -P permet de choisir un port externe automatiquement

```
docker run -p 8080:80 --name php7 php:7-apache
docker run -p 8081:80 --name php8 php:8-apache
docker run -p 53:53/udp bind
docker run -P php:8-apache
```

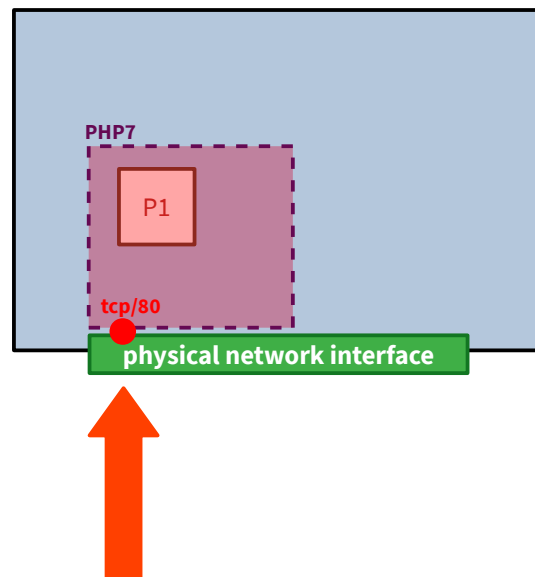


Containers - Docker

Réseaux host

- Permet de lier le processus du container à l'interface de l'hôte.
- Utile pour les services utilisant du trafic broadcast ou multicast (non routable).
- Impossible d'exécuter deux containers utilisant les mêmes ports.

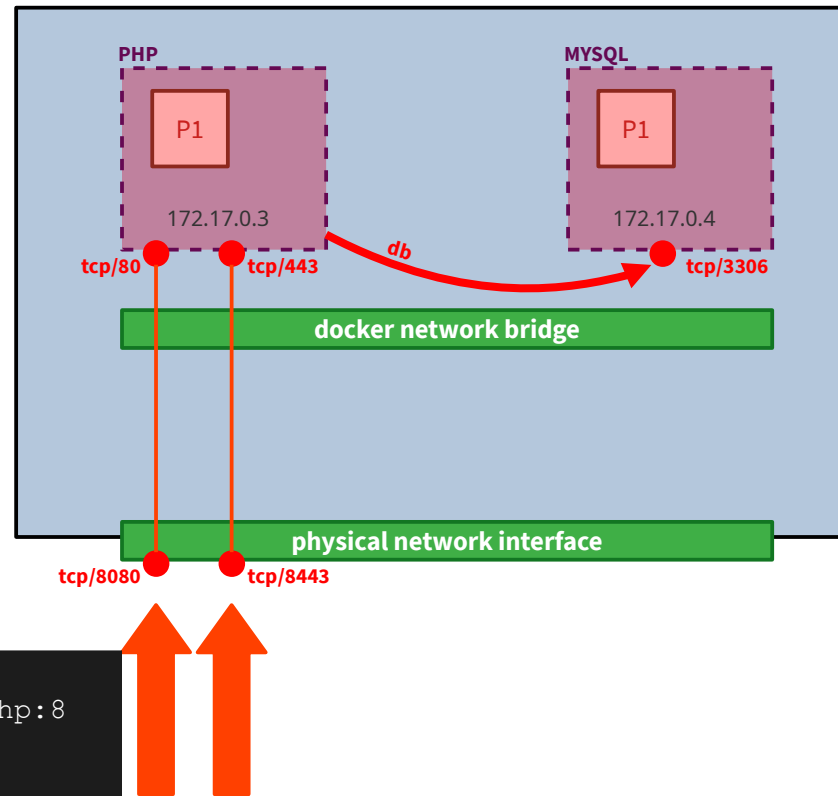
```
docker run --network host --name php7 php:7-apache
```



Containers - Docker

Réseau – communication inter-container

- Les containers dans un même réseau (bridge) peuvent communiquer directement.
- Adressage IP des containers par DHCP.
- Il est difficile de prévoir l'adresse IP d'un container.
- Un serveur DNS interne à Docker permet de trouver l'IP des autres containers du réseau uniquement s'ils ont été déclarés (linking)



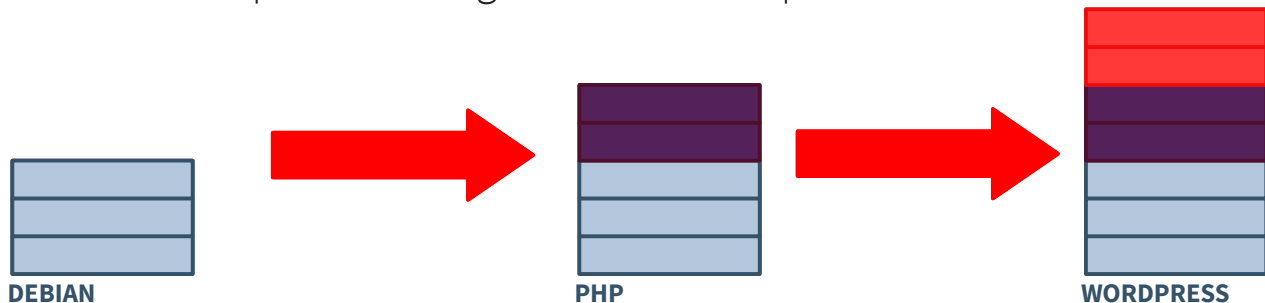
```
docker run -d --name MYSQL mysql
docker run -d -p 8080:80 -p 8443:443 --link MYSQL:db --name PHP php:8
```


Docker – Création d'images

Containers - Docker

Les images Docker sont créées en se basant sur d'autres images.

- L'image initiale (distribution Linux) est préparée de manière spécifique par les mainteneurs de la distribution.
- Une image peut réutiliser les couches d'une autre image et en ajouter.
- Les images peuvent être construites localement.
- Les images sont identifiées par un UUID généré automatiquement.



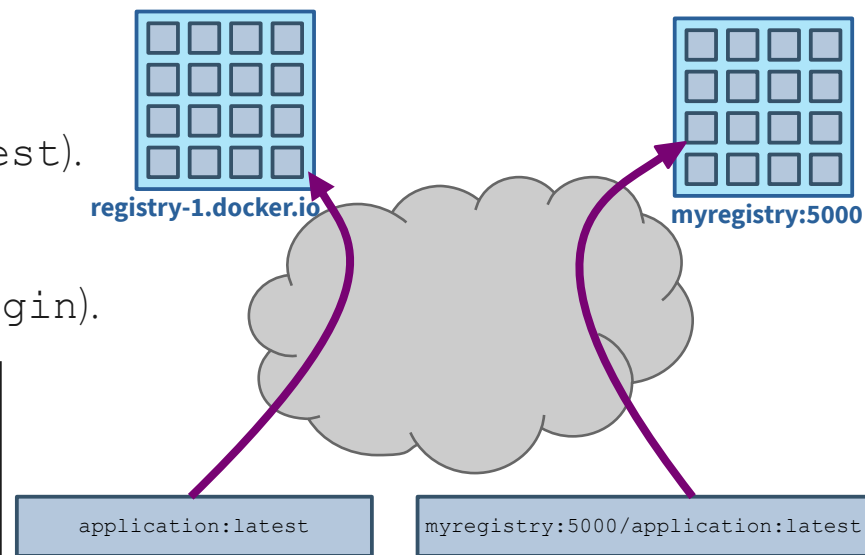
Containers - Docker

Registry Docker

adresse_registry/nom_depot:tag

- Permet de stocker des images à l'image des dépôts git.
- Registry officiel : Docker Hub (<https://hub.docker.com>).
- L'UUID identifie l'image de manière unique.
- Le nom de l'image indique le serveur, l'image et un tag (latest).
- La même image peut être taguée avec plusieurs noms.
- Certains dépôts nécessite une authentification (docker login).

```
docker login myregistry:5000
docker pull application:latest
docker tag application:latest myregistry:5000/application:latest
docker tag application:latest application:1.0
docker push myregistry:5000/application:latest
docker push application:1.0
```

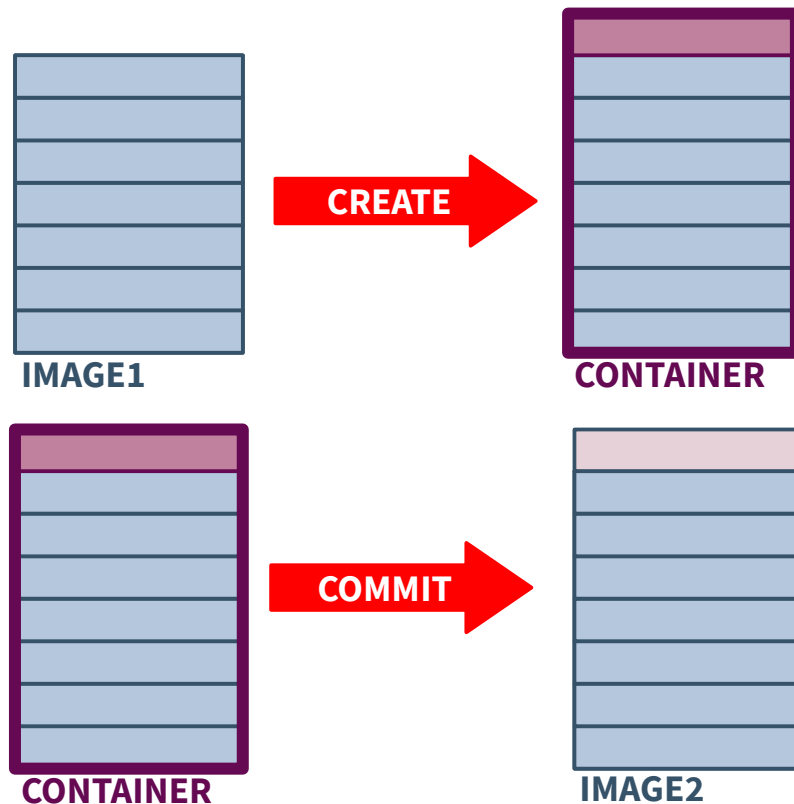


Containers - Docker

Une image peut être créée en commitant un container.

- La couche éphémère devient alors une couche composant l'image créée.
- Cela valide les modifications apportées dans le container.
- Équivalent d'un commit avec `git`.
- Modifications difficilement reproductibles (automatisation compliquée)

```
docker run -d --name webapp httpd
docker exec -it webapp apt-get install -y unzip
docker commit webapp myregistry:5000/application:latest
docker push myregistry:5000/application:latest
```



Containers - Docker

Utilisation d'un fichier Dockerfile

- Fichier décrivant comment construire une image.
- S'appuie sur une image existante.
- Reproductible facilement → automatisation.
- Optimisation pour limiter la taille de l'image et le nombre de couches.
- Mise en cache des opération effectuée au build pour accélérer la construction.
- Référentiel :

<https://docs.docker.com/reference/dockerfile/>

```
FROM debian:buster-slim

# prevent Debian's PHP packages from being installed
# https://github.com/docker-library/php/pull/542
RUN set -eux; \
    { \
        echo 'Package: php*'; \
        echo 'Pin: release *'; \
        echo 'Pin-Priority: -1'; \
    } > /etc/apt/preferences.d/no-debian-php

# dependencies required for running "phpize"
# (see persistent deps below)
ENV PHPIZE_DEPS \
    autoconf \
    dpkg-dev \
    file \
    g++ \
    gcc \
    libc-dev \
    make \
    pkg-config \
    re2c
```

```
docker build -t application:latest .
```

Containers - Docker

Dockerfile - FROM

- Permet de préciser l'image de base à utiliser pour la construction de l'image.
- Possibilité de positionner plusieurs FROM pour définir plusieurs étape à la construction (build stage)
- Possibilité de nommer le contexte et de l'utiliser comme source dans un autre FROM

```
FROM php:8-apache AS source
FROM source
```

```
FROM php
FROM php:8-apache
```

```
FROM golang:1.21 AS build
WORKDIR /src
COPY <<EOF ./main.go
package main
import "fmt"
func main() {
    fmt.Println("hello, world")
}
EOF

RUN go build -o /bin/hello ./main.go

FROM scratch
COPY --from=build /bin/hello /bin/hello
CMD ["/bin/hello"]
```

Containers - Docker

Dockerfile – RUN

- Exécute des commandes dans le contexte de build.
- Exécuté pendant le build, pas à la création / exécution du container
- Chaque RUN crée une nouvelle couche à l'image.
- Enchaînement des commandes avec && et \.
- Possibilité d'utiliser les heredocs (<<EOF)
- Permet des montage au moment de l'exécution (--mount)

```
FROM php:8-apache
RUN apt-get update && \
    apt-get install -y unzip libcurl4 libcurl4-openssl-dev wget msmtplib mailutils && \
    docker-php-ext-install -j$(nproc) mysqli curl

RUN <<EOF
apt-get install zip
EOF
```

Containers - Docker

Dockerfile – COPY / ADD

- Permet de copier des fichiers de l'hôte vers l'image.
- ADD permet d'utiliser des URL en source et de décompresser automatiquement les tar, gzip, bzip2 et xz.
- Possibilité d'utiliser les patterns pour la source (*, ?...)
- --chown et --chmod pour modifier les permissions au moment de la copie.
- --from pour copier depuis un autre contexte, une autre image
- Le fichier .dockerignore permet de ne pas inclure des fichiers dans le build.

```
COPY ./*.sh /bin/  
ADD https://dl.k8s.io/release/v1.30.0/bin/linux/amd64/kubectl /bin/  
ADD --chown=www-data:www-data ./app.tar /var/www/html/  
COPY --from=nginx:latest /etc/nginx/nginx.conf /etc/nginx/
```


Containers - Docker

Dockerfile – WORKDIR

- Permet de se déplacer dans l'arborescence de l'image.
- Équivalent de la commande `cd`.
- Permet d'utiliser des chemins relatifs dans les commande RUN, COPY, ADD, ENTRYPOINT, CMD qui suivent.
- Par défaut le dossier courant est `/`.
- Définit le dossier par défaut lors d'une connexion sur un terminal du container.

```
WORKDIR /var/www/html  
COPY *.html .  
ADD script.sh .  
ENTRYPOINT script.sh
```

Containers - Docker

Dockerfile – ENV / ARG

- Permet de définir une variable d'environnement utilisable dans le contexte de build.
- ENV rend la variable aussi disponibles dans le container créé à partir de l'image contrairement à ARG.
- Il est possible de les surcharger avec :

```
docker run --env VARIABLE=VALUE --build-arg VARIABLE2=VALUE
```

```
FROM ubuntu
ENV APP_DIRECTORY="/app"
ENV DATADIR="${APP_DIRECTORY}/data"
RUN mkdir -p ${DATADIR}
COPY --chmod=755 script.sh ${APP_DIRECTORY}/
CMD ${APP_DIRECTORY}/script.sh
```

script.sh

```
#!/bin/bash
while true; do
    date > $DATADIR
    sleep 1
done
```

Containers - Docker

Dockerfile – EXPOSE

- Permet de spécifier à docker que le processus dans le container écoute sur les port précisés.
- Le paramètre `--port` / `-p` permet de surcharger ces valeurs.
- Donne une indication aux utilisateurs de l'images sur les ports à utiliser.
- Le paramètre `-P` créé une redirection automatique de tous ces ports exposés.

```
EXPOSE 80  
EXPOSE 53/udp
```

```
docker run -d -P image  
docker ps
```

```
PORTS  
0.0.0.0:32768->53/udp, :::32768->53/udp, 0.0.0.0:32768->80/tcp, :::32768->80/tcp
```

Containers - Docker

Dockerfile – VOLUME

- Permet de déclarer un volume qui sera créé automatiquement à la création du container (sans -v)
- Le volume est créé et contient tout ce qui est présent dans le dossier dans l'image.
- Le nom du volume est un identifiant unique difficile à manipuler.
- -v permet de préciser quel volume monter et ne nécessite pas la clause VOLUME dans le Dockerfile

```
VOLUME /app/data  
VOLUME ["/app/data"]
```

Containers - Docker

Dockerfile – ENTRYPOINT / CMD

- Permet de définir ce qui doit être exécuter à la création du container.
- Il faut au moins un CMD ou un ENTRYPOINT.
- CMD peut être utilisé pour définir des arguments de l'ENTRYPOINT à l'exécution du container.
- CMD écrasé par les arguments à l'exécution du container contrairement à l'ENTRYPOINT.
- `exec` permet un arrêt du container par `docker stop`

```
ENTRYPOINT /app/script.sh
ENTRYPOINT exec /app/script.sh
ENTRYPOINT /app/script.sh dev
ENTRYPOINT ["/app/script.sh", "prod"]
CMD /app/script.sh
CMD /app/script.sh dev
CMD ["/app/script.sh", "prod"]
```

```
FROM ubuntu
COPY --chmod=755 script.sh /bin/
ENTRYPOINT script.sh
CMD prod
```

```
docker run test
Version prod
```

```
docker run test dev
Version dev
```

Containers - Docker

Exemple de fichier Dockerfile

```
FROM php:8-apache
RUN apt-get update && \
    apt-get install -y unzip libcurl4 \
    libcurl4-openssl-dev wget msmtplib mailutils && \
    docker-php-ext-install -j$(nproc) mysqli curl

ARG version=1.4.3

RUN wget -O /tmp/easyappointments-${version}.zip https://github.com/alextselegidis/easyappointments/releases/download/${version}/easyappointments-${version}.zip

WORKDIR /var/www/html/
RUN unzip /tmp/easyappointments-${version}.zip && \
    rm -rf /tmp/easyappointments-${version}.zip && \
    chown www-data:www-data -R .

ADD --chmod=600 --chown=www-data:www-data msmtplib /etc/msmtplib

RUN echo "sendmail_path=/usr/bin/msmtplib -t" >> $PHP_INI_DIR/conf.d/php-sendmail.ini
```

Docker Compose

Containers - Docker

Spécifier l'ensemble des paramètres pour tous les containers d'une application peut vite devenir fastidieux.

- Docker Compose permet de définir tous les containers avec leurs détails dans un fichier YAML.
- Commande Python `docker-compose` puis commande Go (`docker compose`) à partir de 2020.
- Le fichier YAML peut aussi être utilisé avec l'orchestrateur Docker Swarm en utilisant quelques clauses spécifiques.
- Tous les paramètres utilisables avec la commande `docker run` sont supportés par docker compose.
- Un réseau docker spécifique est créé pour les containers associés dans le fichier.
- Les containers d'un même fichier peuvent utiliser leurs noms pour communiquer.

Containers - Docker

Exemple de fichier docker-compose.yml

```
services:
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    ports:
      - "8080:80"
    depends_on:
      - mysql
    environment:
      - PMA_HOST=mysql
  mysql:
    image: mysql:8
    volumes:
      - mysql-data:/var/lib/mysql
    environment:
      - MYSQL_ROOT_PASSWORD=root

volumes:
  mysql-data:
```

Containers - Docker

Commandes

```
# Création / recréation des containers
docker compose up -d

# Arrêt et suppression des containers
docker compose down

# Liste des containers associés au fichier
docker compose ps

# Logs des containers ou d'un container
# spécifique si spécifié
docker compose logs

# Build des images (clauses build et image)
docker compose build

# Build de l'image pour le service mysql
docker compose build mysql
```

```
# Récupération des images concernées
docker compose pull

# Envoi de l'image associée au service mysql
docker compose push mysql

# Utilisation d'un fichier yaml spécifique
docker compose -f docker-compose-dev.yml up -d

# Utilisation d'un fichier d'environnement autre
docker compose --env-file .env-dev up -d
```

Containers - Docker

Fichier de variables d'environnement

- Il est possible d'utiliser un fichier externe de variables d'environnements.
- Par défaut `.env` dans le même dossier que le fichier YAML.
- Peut être spécifié dans la commande docker compose (`--env-file`) ou dans le fichier YAML (`env_file:`) pour utiliser un fichier ayant un autre nom.
- Utilisation de la syntaxe `${variable}` dans le fichier YAML.

`.env`

```
TAG=dev  
HTTP_PORT=8080
```

`docker-compose.yml`

```
services:  
  phpmyadmin:  
    image: phpmyadmin/phpmyadmin:${TAG}  
    ports:  
      - "${HTTP_PORT}:80"
```

Mise en production

sans orchestrateur

Containers - Docker

Il est possible de mettre en production des applications tournant avec Docker sans passer par un orchestrateur de container (Docker Swarm ou Kubernetes).

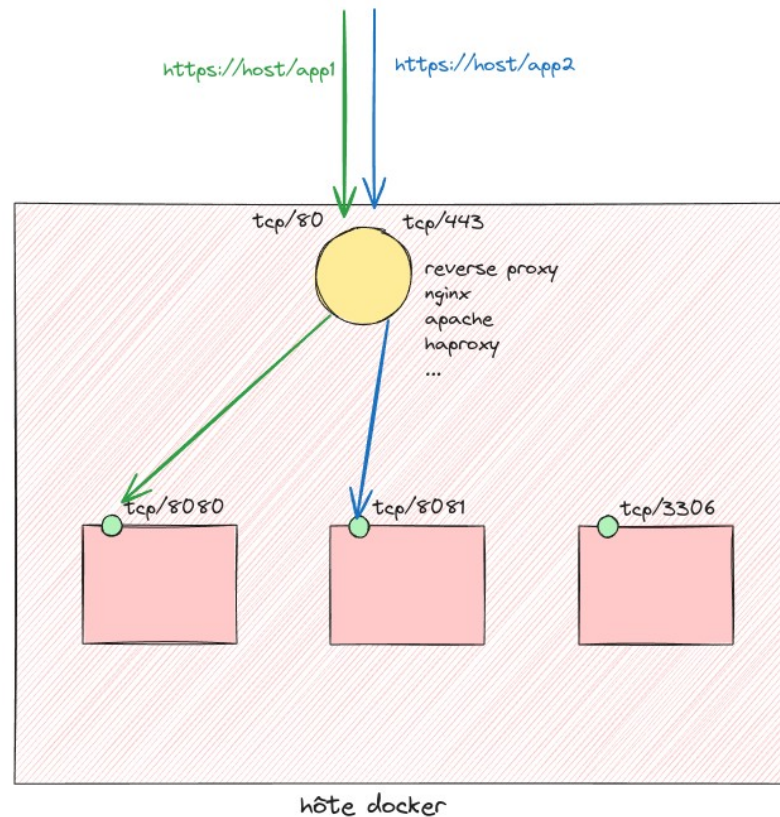
- Démarrage automatique des containers avec le systèmes (restart policy).
- Possibilité d'effectuer une sauvegarde des données (volumes).
- Utilisation de ports par défaut, notamment pour les applications Web.
- Mise en place de certificats TLS pour sécuriser les flux.

Pour les applications Web, il semble indispensable de passer par un **reverse proxy** pour la mise en production.

Containers - Docker

Utilisation d'un reverse proxy

- nginx, apache, haproxy, traefic...
- Possibilité de gérer des certificats.
- Possibilité de le faire tourner dans un container.
- Possibilité de faire de la répartition de charge.
- Définition du routage par url d'accès : IP, nom d'hôte, chemin...



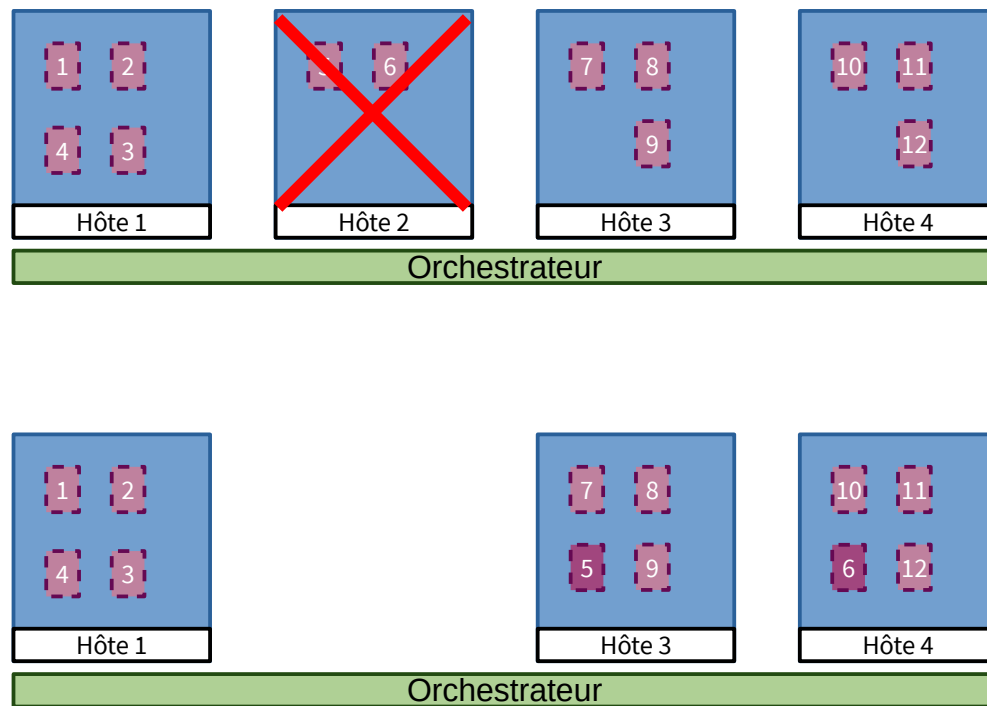
Orchestrateurs de containers

Mettre en production des containers

Containers - Orchestrateurs

Rôles des orchestrateurs de containers

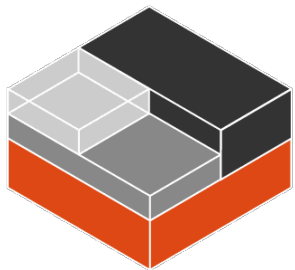
- Centraliser la gestion des containers
- Gérer la tolérance à la panne des hôtes
- Gérer la répartition de charge (scaling)
- Gérer le déploiement / mise à jour des containers
- Gérer l'accessibilité des containers
- Gérer la connectivité des containers entre les hôtes



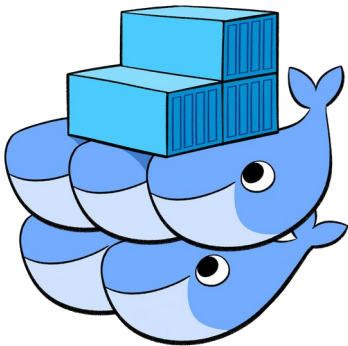
Containers - Orchestrateurs

Technologies

- LXD / Incus : orchestrateur de containers LXC (créé par Canonical)
- Docker Swarm : orchestrateur natif de containers Docker
- Kubernetes : orchestrateur de containers Docker, Rkt...



LXD



kubernetes

Docker Swarm

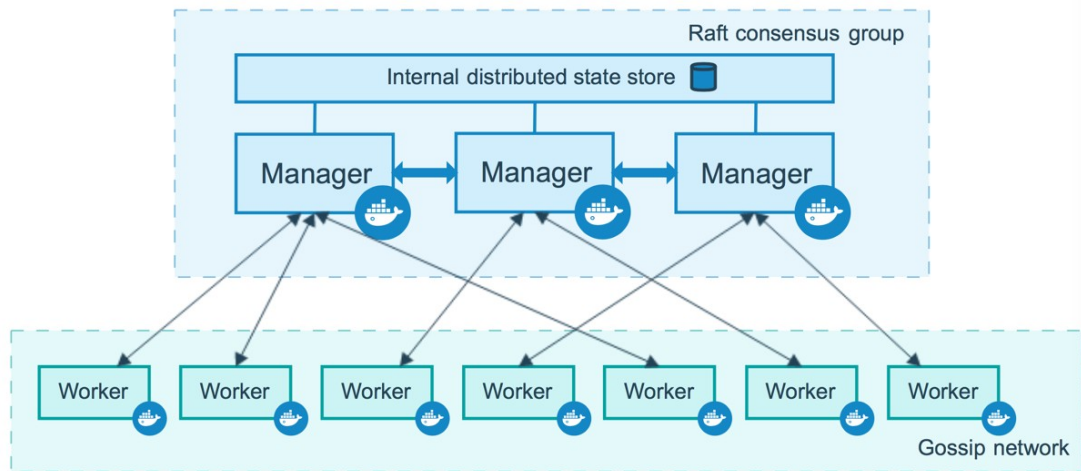
L'orchestrateur intégré à Docker

Containers - Orchestrateurs

Docker Swarm

- Intégré à Docker CE
- Simple à apprendre
- Utilise la commande docker CLI
 - `docker swarm init`
 - `docker swarm join`
- Même syntaxe que docker run
- Service : désigne un container pouvant disposer de plusieurs répliquas
- Routage du trafic entre les nœuds
- Déploiement des applications par un fichier YAML Docker Compose

- Nodes Manager : gestion des workers
- Nodes Worker : exécution des containers



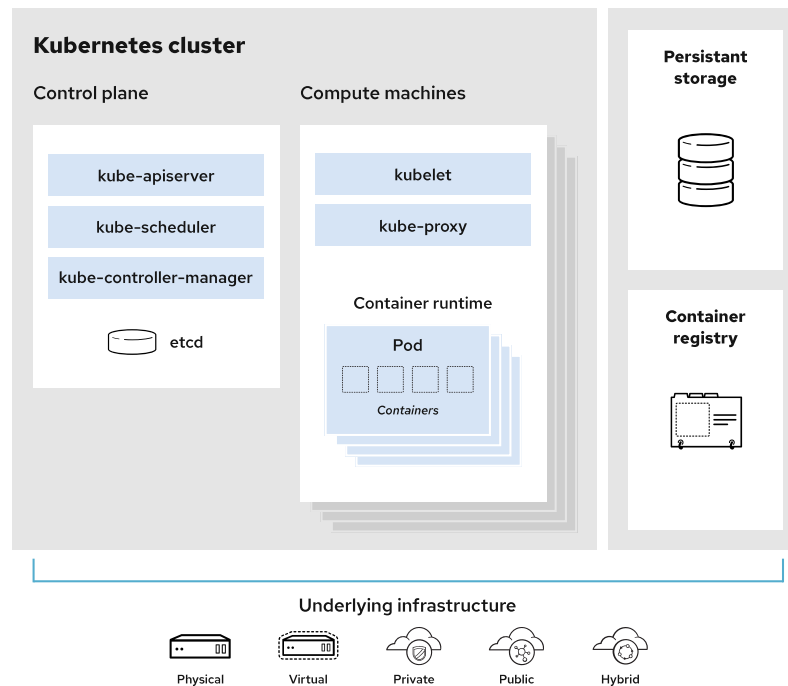
Kubernetes

L'orchestrateur de containers de référence

Docker - Orchestrateurs

Kubernetes

- Développé par Google
- Open-source
- Largement déployé
- Architecture complexe
- Indépendant de la technologie de conteneurisation
- POD : containers partageant le même stockage
- Commande `kubectl`
- Objets deployment, service, cronjob, persistent storage, config map, ...



To be containerized...